

Using the Ackermann Function

The *Ackermann function* is named after its discoverer, Wilhelm Ackermann. A student of mathematician David Hilbert (whose Hilbert curve fractal we discuss in Chapter 9), Ackermann published his function in 1928. Mathematicians Rózsa Péter and Raphael Robinson later developed the version of the function featured in this section.

While the Ackermann function has some application in advanced mathematics, it is mostly known for being an example of a highly recursive function. Even slight increases to its two integer arguments cause a large increase in the number of recursive calls it makes.

The Ackermann function takes two arguments, m and n , and has a base case of returning $n + 1$ when m is 0. There are two recursive cases: when n is 0, the function returns `ackermann( $m - 1$ , 1)`, and when n is greater than 0, the function returns `ackermann( $m - 1$ , ackermann( $m$ ,  $n - 1$ ))`. These cases likely aren't meaningful to you, but suffice it to say, the number of recursive calls the Ackermann function makes grows quickly. Calling `ackermann(1, 1)` results in three recursive function calls. Calling `ackermann(2, 3)` results in 43 recursive function calls. Calling `ackermann(3, 5)` results in 42,437 recursive function calls. And calling `ackermann(5, 7)` results in . . . well, actually I don't know how many recursive function calls, because it would take several times the age of the universe to calculate.

Let's answer the three questions we ask when constructing recursive algorithms:

What is the base case? When m is 0.

What arguments are passed to the recursive function call? Either m or $m - 1$ is passed for the next m parameter; and 1, $n - 1$, or the return value of `ackermann( $m$ ,  $n - 1$ )` is passed for the next n parameter.

How do these arguments become closer to the base case? The m argument is always either decreasing or staying the same size, so it will eventually reach 0.

Here is an *ackermann.py* Python program: